

CSCI 447 Project 1 K - Nearest Neighbor Algorithms for Classification and Regression

Zach Reller

*Gianforte School of Computing
Montana State University
Bozeman, MT*

ZACHARY.RELLER@MONTANA.STUDENT.EDU

Gaige Bache

*Gianforte School of Computing
Montana State University
Bozeman, MT*

GAIGEBACHE@MONTANA.STUDENT.EDU

Abstract

This project evaluates several K-Nearest Neighbor (KNN) variants across three classification and three regression datasets. We tested standard KNN with Edited KNN and Condensed KNN against Null Models. We then compared whether hyperparameter tuning would improve the KNN variants performances. The models were evaluated using a 5x2 cross-validation harness. In our tests we found the KNN variants outperformed the Null Models on classification tasks and had varying performances when working on regression datasets. Hyperparameter tuning returned mixed results, substantially improving performance on some datasets and reducing performance on others. Edited KNN and Condensed KNN each showed different performances depending on the dataset, showing that dataset reduction doesn't consistently preserve predictive performance. Our results indicate that that the effectiveness of KNN and its variants depends on the characteristics of the dataset.

1 Problem statement and Hypothesis

The goal of this project was to evaluate and compare four types of K-Nearest Neighbor (KNN) algorithms across multiple numerical and categorical datasets. We compared the performance of Null, standard KNN, condensed KNN, and modified KNN as each type of algorithm predicted both regression as well as classification.

Hypothesis: We believe that across all datasets, the K-Nearest Neighbors (KNN) models will outperform the null models because KNN uses information from the training data to make predictions rather than relying on a baseline prediction. We also believe that the models that undergo hyperparameter tuning will finish with improved results compared to the standard models. We think this because the hyperparameterization process finds the optimal running parameters for the models on an individual level, meaning the models will have the best possible setting to run their tests off of. Finally, we believe that the edited and condensed KNN models will have differing performance across the datasets. We think this is due to the different process the two models have of reducing the data that forms the prediction.

2 Experimental approach and explanation of the project design

2.1 Datasets

We have six datasets that are broken up in half with three aimed at classification and three that are predicted by regression models. In the car dataset, we are looking to predict what quality class the car will fall into, while dealing with categorical variables, we did not do any preprocessing. Our next classification dataset is the house votes dataset, where we are looking to predict the political party of a given voter. The dataset has sixteen key votes, while preprocessing our data we changed any "?" in the dataset into an abstain vote. This was done as the original data set lists any value that is not a yes or a no as a ?, it doesn't make sense to drop the values as it would shrink the dataset significantly, also classifying it as a yes or no would be incorrect, so treating it as a third category keeps its signal that it was neither a yes or no. Our last dataset that uses a classifier model is the Wisconsin breast cancer dataset, it holds numerical values and looks to classifier whether or not a tumor is benign or malignant. There are sixteen missing values in the dataset, to account for this we remove the entire row from the dataset as it doesn't impact the total amount of instances severely.

The other three datasets use regression models to give predictions. We first look at the machine dataset, which wants to predict the published relative performance or prp of a cpu. In other words we want to see how well a cpu works based on the quantitative components of the dataset. We remove the vendor name and model name from the dataset as they give no relevant information to help us predict the prp. We also remove the estimated relative performance as it gives us no extra information on the performance of the cpu. We then look at the abalone dataset where we attempt to predict the age of the abalone by inspecting how many rings it has. We use the datasets numerical values to find the predicted age, we also one hot encode the gender to keep its influence on the abalones number of rings. Our final dataset for the project is the forest fire dataset and looks at how different numerical factors impact the total area a wildfire burns. To fix the skew that can occur when finding the total area, we perform a log transform to normalize the data. This dataset also includes the month and day the fires took place, since these values are cycles rather than a linear quality, we label-encode them as integers. We then compute distances between them using wrap-around metrics, so months like December and January are treated as one month apart, instead of eleven.

2.2 Feature Representation and Normalization

To normalize our data and make sure all features are treated equally we use min max normalization for our quantitative features. This process scales each of the quantitative features on a scale from 0 to 1. Without this normalization, features with larger numerical ranges would have a disproportionate influence on the KNN algorithm leading to biases when creating a prediction.

For our datasets that utilize categorical features, we use the Value Difference Metric (VDM) to determine the distance between each of the values under each feature. VDM uses the distribution of target classes that are associated with each categorical value to determine how similar two values are. This allows us to use datasets with categorical values

in the same distance based KNN algorithms without the need to assign arbitrary numerical values to each of the categories.

2.3 Distance Metrics

We used Minkowski's general distance formula as a distance metric for our KNN algorithms. In our tests, we use both Manhattan ($p=1$) and Euclidean ($p=2$) distance metrics to calculate distances and make our predictions. As previously stated, we use the Value Distance Metric (VDM) to find the distance between categorical values.

2.4 Cross-Validation Design

We use 5 x 2 cross validation to evaluate the performance of our algorithms. In 5 x 2 cross validation, we evenly split the data into two random subsets, one for training and the other for testing. We then reverse the roles of the subsets, giving us two evaluations for each repetition. We repeat this process five times, giving us a total of ten train-test evaluations. Using multiple splits allows us to obtain a more reliable estimate of our models performance and account for any variation caused by the splits.

In our experiments, we use the same splits for each dataset across all algorithms in the project. This ensures that each algorithm is being evaluated on the same training and testing splits across each fold. After the ten evaluations, we calculate the mean and standard deviation of the results. Using identical splits allows us to compare the performance of different models under the same conditions.

2.5 Hyperparameter Tuning

We use hyperparameter tuning to help determine what parameters we can assign that will provide the best performance for each of our models. The hyperparameters we tune for include the number of neighbors (k), if we are using Manhattan or Euclidean distance (p), the categorical weighting parameter (γ), and the error threshold ϵ . The k and p parameters are used by our KNN-based algorithms, while γ controls the contribution of the categorical distances, and ϵ is used as an error threshold for the edited and condensed regression methods.

The hyperparameters are selected and tested separately for each of the datasets on each outer training set. For each of the outer folds for the 5 x 2 cross validation, we perform an additional five fold cross validation on the training data to evaluate different hyperparameter combinations. We select the combination of parameters that results in the lowest mean squared error for regression models or lowest classification error for the classification models. After parameters are selected, the algorithm is then retrained using the outer training set, then tested on the outer test set.

This allows us to select the best possible hyperparameters, then test those on an unbiased evaluation. Because the optimal parameters can vary between cross-validation folds and datasets, the chosen hyperparameters are determined independently for each of the outer evaluations.

Algorithm	k	p	γ	ϵ
KNN Classification	3, 5, 7	1, 2	X	X
Edited Classification	3, 5, 7	1, 2	X	X
Condensed Classification	3, 5, 7	1, 2	X	X
KNN Regression	3, 5, 7	1, 2	1, 5, 10, 20	X
Edited Regression	3, 5, 7	1, 2	1, 5, 10, 20	25, 50, 100, 200
Condensed Regression	3, 5, 7	1, 2	1, 5, 10, 20	25, 50, 100, 200

Table 1: Tested Hyperparameter Values Across Model Type

2.6 Evaluation Metrics

To evaluate the performance of our algorithms, we use classification error and mean squared error. To check the accuracy of our regression models we use mean squared error, which averages the squared distance between the predicted and actual values:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

A lower mean squared error indicates the models prediction is closer to the actual values as the errors being squared means larger prediction errors have a greater influence.

For our classification models, we use classification error to evaluate, this measures the proportion of predictions that are incorrectly classified:

$$ClassificationError = \frac{1}{n} \sum_{i=1}^n 1(y_i \neq \hat{y}_i)$$

A lower classification error means that the model made fewer incorrect classification predictions.

3 Explanation of algorithms and how we implemented them

3.1 Distance Metrics

For numeric features, we use the Minkowski distance:

$$D_{num}(x, y) = \left(\sum_{k=1}^d |x_k - y_k|^p \right)^{1/p}.$$

For categorical features, we use the Value Difference Metric (VDM). For feature f with value v_i , let C_i denote the number of training instances with $f(x) = v_i$, and $C_{i,a}$ the number of those instances belonging to class a :

$$\delta(v_i, v_j) = \sum_{a=1}^{numclasses} \left| \frac{C_{i,a}}{C_i} - \frac{C_{j,a}}{C_j} \right|^p, \quad D_{cat}(x, y) = \left(\sum_{k=1}^d \delta(x_k, y_k) \right)^{1/p}.$$

When a dataset contains both feature types, the two distances are summed without taking the individual p -th roots, since doing so does not affect neighbor ordering:

$$D(x, y) = D_{num}(x, y) + D_{cat}(x, y).$$

For the Forest Fires dataset, the `month` and `day` features are cyclical and use

$$D_{cyc}(a, b) = \min(|a - b|, L - |a - b|),$$

with $L = 12$ for month and $L = 7$ for day.

3.2 Null Models

For classification, prediction is decided by the most common class in the entire dataset. If there is a tie for the most common class, then a class is randomly selected from the list of most common classes:

$$\hat{y}_{null} = \arg \max_{c \in C} \sum_{i=1}^n 1(y_i = c)$$

For regression, we take the mean of the training target values and return that as the regression prediction:

$$\hat{y}_{null} = \frac{1}{n} \sum_{i=1}^n y_i$$

3.3 k-Nearest Neighbors

For classification, prediction is by plurality vote among the k nearest neighbors $\mathcal{N}_k(x)$, with ties broken uniformly at random:

$$\hat{y} = \arg \max_{c \in C} \sum_{i \in \mathcal{N}_k(x)} 1(y_i = c).$$

For regression, prediction uses a Gaussian (RBF) kernel-weighted average:

$$K(x, x_q) = \exp(-\gamma \|x - x_q\|_2^2), \quad \hat{y}(x_q) = \frac{\sum_{i \in \mathcal{N}_k(x_q)} K(x_i, x_q) y_i}{\sum_{i \in \mathcal{N}_k(x_q)} K(x_i, x_q)}.$$

3.4 Edited k -Nearest Neighbors

Our version of Edited k -NN reduces the training set D by removing the instances that are misclassified by their nearest remaining neighbor, using $k = 1$ during the reduction process itself. We implement the standard Wilson editing rule (Wilson, 1972), which removes points that disagree with their neighborhood, as opposed to a converse variant that removes correctly-classified (redundant) points instead.

Classification. For each instance $x_i \in D$, define its leave-one-out nearest-neighbor prediction as

$$\hat{y}_i = y_{j^*}, \quad j^* = \arg \min_{j \neq i, x_j \in D} \text{dist}(x_i, x_j).$$

Instance x_i is removed if it is misclassified by its nearest neighbor:

$$\text{remove } x_i \iff \hat{y}_i \neq y_i.$$

To prevent a minority class from being eliminated entirely, we retain at least one representative of each class even when it would otherwise be marked for removal.

Regression. The same leave-one-out prediction is used, but correctness is defined relative to an error threshold ε rather than an exact label match:

$$\text{remove } x_i \iff |\hat{y}_i - y_i| > \varepsilon.$$

Similarly, we impose a minimum retained set size, removing only the worst-offending points once further removal would fall below this floor.

Convergence. In both cases, this process is repeated over successive passes of the shrinking dataset:

$$D^{(t+1)} = D^{(t)} \setminus \{x_i \in D^{(t)} :$$

$x_i \text{ is incorrect}\}, \text{ until a full pass removes no further points, i.e. } D^{(t+1)} = D^{(t)}.$ Once editing converges, final predictions on the test set are made using the tuned value of k (not $k = 1$) applied to the reduced set $D^{(t+1)}$.

3.5 Condensed k -Nearest Neighbors

Condensed k -NN instead builds up a minimal representative subset $Z \subset D$, starting from a single randomly selected instance and adding only those points that are not already correctly handled by the current Z , again using $k = 1$ during construction. We implement the standard Hart condensing algorithm (Hart, 1968).

Classification. For each instance $x_i \notin Z$, define its nearest-neighbor prediction *within the current Z* :

$$\hat{y}_i = y_{j^*}, \quad j^* = \arg \min_{x_j \in Z} \text{dist}(x_i, x_j).$$

Instance x_i is added to Z if it is misclassified by its nearest neighbor in Z :

$$\text{add } x_i \text{ to } Z \iff \hat{y}_i \neq y_i.$$

Unlike our Edited k -NN implementation, condensing imposes no explicit safeguard against eliminating a minority class from Z ; a class is retained only insofar as at least one of its instances is needed to correctly classify the rest of the training set.

Regression. The same nearest-neighbor-in- Z prediction is used, with the ε -threshold rule in place of exact label matching:

$$\text{add } x_i \text{ to } Z \iff |\hat{y}_i - y_i| > \varepsilon.$$

Convergence. Z grows across successive passes over the remaining instances not yet included:

$$Z^{(t+1)} = Z^{(t)} \cup \{x_i \notin Z^{(t)} :$$

x_i is incorrect w.r.t. $Z^{(t)}\}, \text{until a full pass adds no further points, i.e. } Z^{(t+1)} = Z^{(t)}.$ As with Edited k -NN, final predictions on the test set are made using the tuned value of k applied to the condensed set $Z^{(t+1)}$.

Sensitivity to initialization. Because Z is built incrementally from a single random starting point and the order in which remaining instances are processed, the resulting condensed set is not unique: different seeds can produce condensed sets of different size and quality. We discuss the practical impact of this sensitivity, and how hyperparameter tuning partially compensates for it.

3.6 Statistical Significance: 5×2 CV Paired t -Test

Following our textbook, we use the 5×2 cross-validation paired t -test to determine whether the difference in performance between two algorithms is statistically significant. Because all algorithms are evaluated using the same 5×2 cross-validation splits, the results from each corresponding train-test evaluation can be paired when comparing two methods.

For each of the five repetitions, we calculate the difference between the scores of the two algorithms for each of the two folds. Let

$$p_i^{(1)} \quad \text{and} \quad p_i^{(2)}$$

represent the performance differences between the two algorithms for the two folds in repetition (i). We calculate the mean difference for each repetition as

$$\bar{p}_i = \frac{p_i^{(1)} + p_i^{(2)}}{2}.$$

The variance estimate for each repetition is calculated as

$$s_i^2 = \left(p_i^{(1)} - \bar{p}_i\right)^2 + \left(p_i^{(2)} - \bar{p}_i\right)^2.$$

The resulting test statistic is used with a t -distribution with 5 degrees of freedom to determine whether the observed difference between the two algorithms is statistically significant. A statistically significant result indicates that the observed difference in performance is unlikely to be explained solely by variation between the cross-validation splits.

4 Experimental results (tables/graphs w/ explanation)

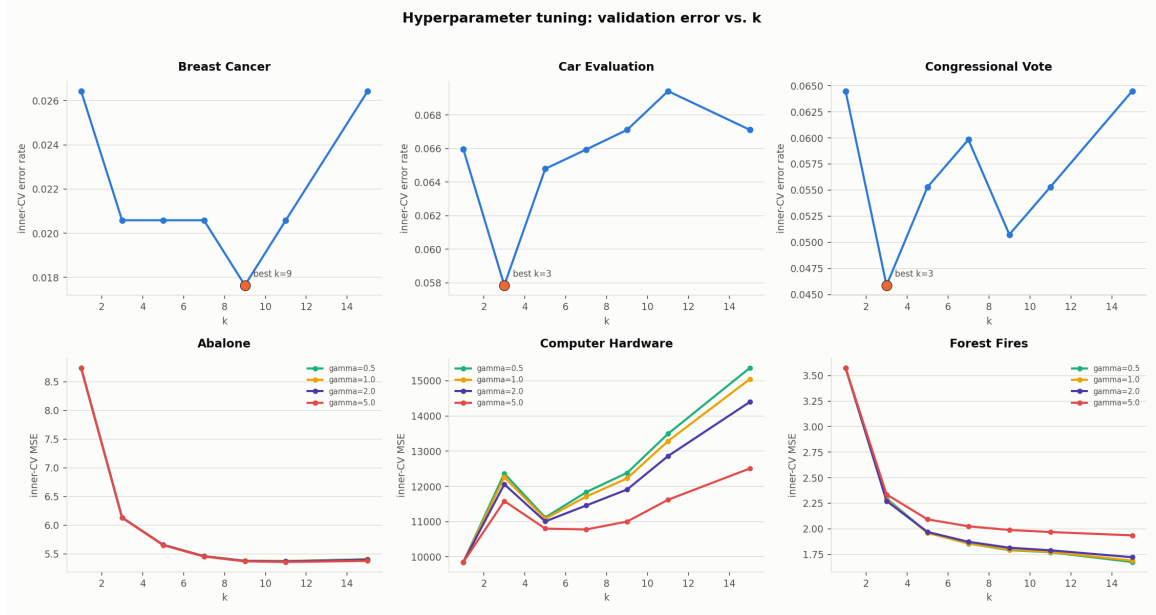


Figure 1: Hyperparameter tuning across all six datasets.

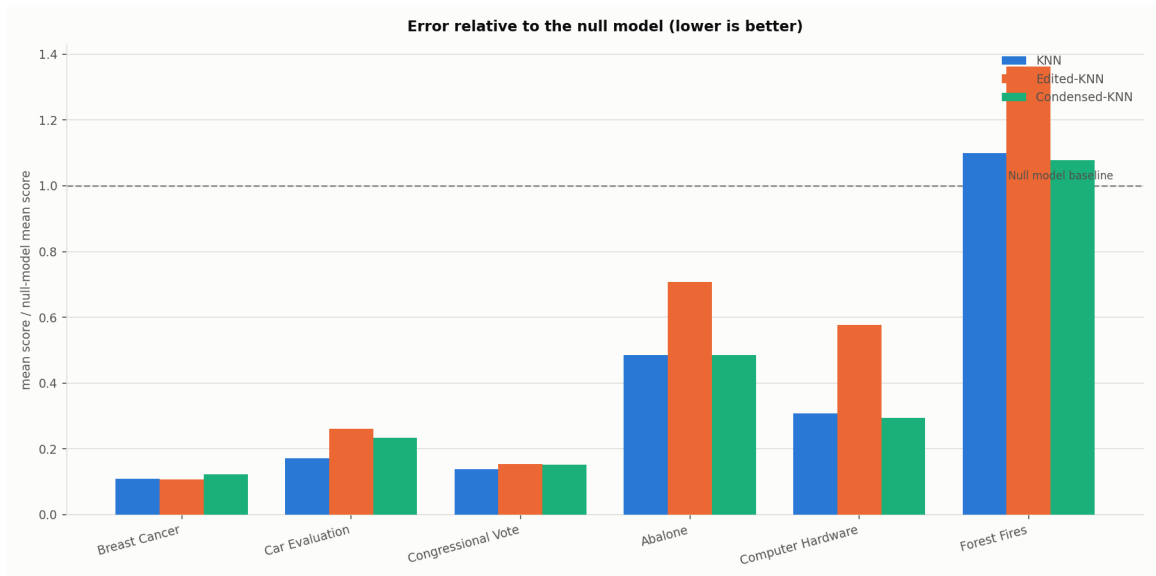


Figure 2: Error of each KNN variant relative to the null model, by dataset.

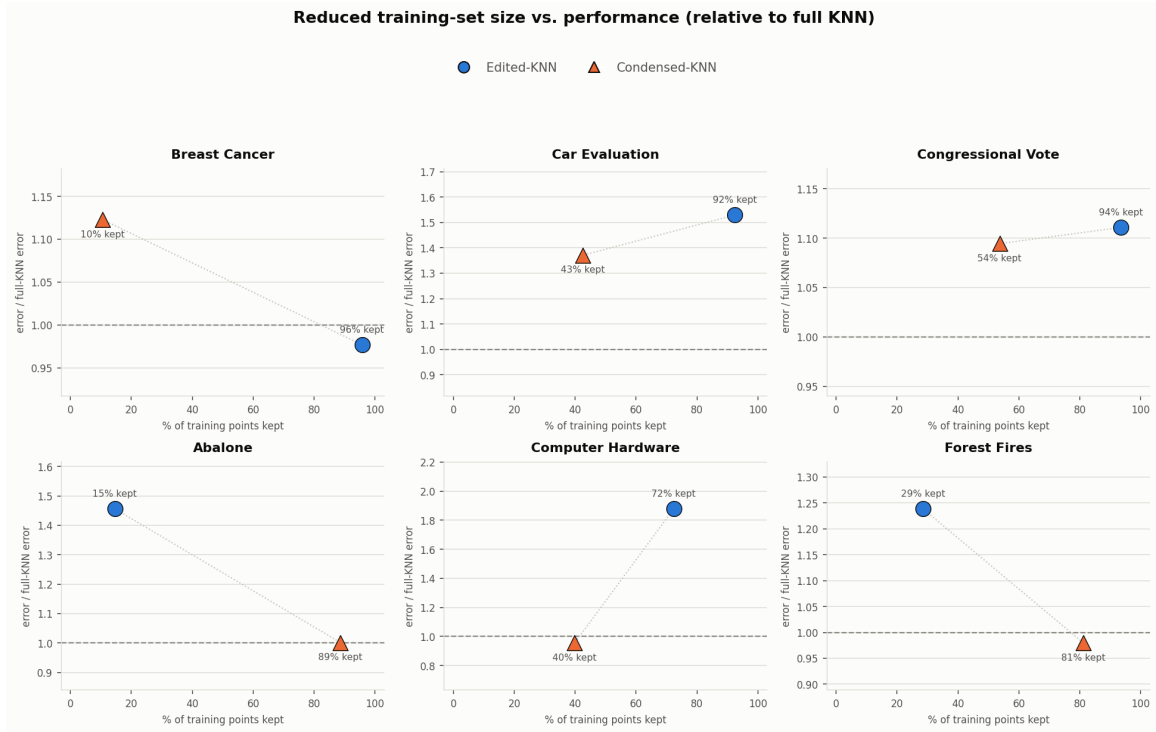


Figure 3: Edited-KNN (blue) vs. Condensed-KNN (orange) reduction size vs. relative error, per dataset.

Dataset	Null	KNN	Tuned KNN	Edited-KNN	Tuned Edited	Condensed-KNN	Tuned Condensed
Abalone	41.3956	66.8204	65.3534	66.8204	54.1841	18101.8336	23770.4336
Computer Hardware	90536.0767	46742.6532	28122.7235	133764.2601	41572.8899	22205.9815	18564.7057
Forest Fires	2.8991	3.7904	4.4810	3.7904	4.4410	340.0930	961.6102

Table 2: Mean MSE across the 5×2 cross-validation evaluations for the regression datasets. Lower values indicate better performance.

Dataset	Null	KNN	Edited-KNN	Tuned Edited	Condensed-KNN	Tuned Condensed
Breast Cancer	0.3499	0.0331	0.0354	0.0351	0.0357	0.0416
Car Evaluation	0.2998	0.0497	0.0856	0.0745	0.2145	0.0841
Congressional Vote	0.3862	0.0575	0.0529	0.0524	0.0749	0.0602

Table 3: Mean classification error across the 5×2 cross-validation evaluations for the classification datasets. Lower values indicate better performance.

4.1 Tuning Results

The hyperparameter tuning process selected different combinations of parameters across datasets and cross-validation folds. This process highlights how the most optimal parameters for each KNN variant is dependent on the characters and splits inside of the dataset.

Tuned KNN Our tuned KNN shows how the selected parameters varied greatly between each of the datasets. In the Computer Hardware dataset, the tuning mainly selected γ 's of 5 or 20, with $p = 1$ and $p = 2$ appearing across many of the folds. In Abalone, the range of γ 's selected included 1, 10, and 20 with k values of 3, 5, and 7 all being selected. On the other hand, Forest Fires primarily selected $\gamma = 1$, while k and p varied between the various folds. This variance across folds and datasets tells us that the optimal KNN parameters were dependent on the characteristics of the datasets rather than one parameter combination across all datasets.

Tuned Edited-KNN The tuned Edited-KNN results show us a more consistent parameter selection across each of the regression datasets. Computer Hardware consistently selected $k = 3, p = 2$ with $\gamma = 20$ in all folds and ϵ as 200 in the majority of folds. Abalone also selected $k = 3, p = 2$ as well as $\epsilon = 25$ in the folds with γ varying between 1 and 20. Forest Fires similarly to some of the other data sets chose $k = 3, p = 2$, and $\epsilon = 25$ across the folds with many different variations for γ . For the classification datasets, smaller k values were selected, specifically $k = 1$ for both the Congressional Vote and Breast Cancer datasets. Overall, Edited-KNN showed significantly more consistency for k, p , and ϵ than for γ .

Tuned Condensed-KNN Our tuned Condensed-KNN models showed much more variation in the selected hyperparameters across both datasets and folds. Each of our regression datasets selected different combinations for k, p, γ , and ϵ , indicating to us that the condensed training sets were sensitive to the hyperparameter configuration. Parameter selections were generally more consistent for the classification datasets. Car Evaluation selected $k = 3$ in all folds and $p = 1$ in all but one fold. The other classification datasets' showed more variation in k , including Breast Cancer selecting values of $k = 3, 5$, and 7. These results show us that the condensed approach didn't have one single consistent optimal parameter combination across the datasets similar to the results from our Tuned Edited-KNN hyperparameterization.

4.2 Interpretation of Model Results

The results show that the performance of the KNN variants varied considerable across the datasets. For the regression datasets, the variants of the KNN models outperformed the Null model for the Computer Hardware and Abalone datasets, however the Null model outperformed all of the KNN models for the Forest Fires dataset. For the categorical datasets, the KNN variants correctly predicted the classes significantly more accurate than the Null Model across all three datasets.

5 Analysis and discussion of results

KNN variants beat out the null model across almost all datasets, the exception to this is the Forest Fires dataset ($p=0.32$). The target is skewed by near-zero burn areas because fires

are unpredictable, with a noisy long tail. This long tail in the dataset may contribute to the regression for the KNN models to have a much harder time to make accurate predictions, making the KNN models less effective. This leads to the distance-based structures being rendered less successful compared to the Null Model. For our classification datasets, the KNN-variation models perform significantly better than the Null Model. One possible explanation for this is these datasets having more consistent relationships across features. This allows for useful information for our KNN distance models to capture more insightful data compared to the regression models. These results support our first hypothesis that the KNN model variations would outperform the null model with the exception of the Forest Fires dataset.

The hyperparameter tuning results showed us that the tuning did not consistently improve the performance of the KNN variants across the datasets. The greatest improvements came in the Computer Hardware dataset, this dataset saw the MSE of the standard KNN drop from 46,742.65 to 28,122.72. It also had the MSE for Edited-KNN go from 133,764.26 to 41,572.89. For the Abalone dataset, we had smaller improvements, the standard KNN MSE dropped from 66.82 to 65.35, while Edited-KNN dropped from 66.82 to 54.18. However, the performance for the regression KNN models dropped for the Forest Fire dataset when the models underwent hyperparameter tuning. For the classification models, the models showed improvement for the Car Evaluation and Congressional votes while the Tuned model for Condensed-KNN performed worse on the Breast Cancer dataset. These results indicate that the characteristics have a large impact on the effectiveness of hyperparameterization for the KNN variations we use in this project.

The results show that the Edited-KNN and Condensed-KNN models behave differently across the datasets. Both models use different approaches to reducing the training dataset, this can lead to differing effects on each models performance for each dataset. Our Edited-KNN models removes elements from the datasets that do not appear to give positive contributions to the model. On the other hand, Condensed-KNN is attempting to retain a smaller subset of training data. Areas where this can impact the results the most is when an Edited model removes an unusual point that can be useful for regression or when a condensed model preserves important boundary points. The difference in strategy is most noticeable in the regression models. On the Abalone dataset, Edited-KNN improves on the standard KNN model, while Condensed-KNN produces substantially higher MSE values. The classification datasets show that Condensed-KNN can reduce the amount of training data while also maintaining performance. An example of this is how Condensed-KNN only kept 10.5% of the original Breast Cancer training set without a significant difference from the standard KNN model ($p = 0.329$). However, on the Car Evaluation dataset, Condensed-KNN performed significantly worse than the standard KNN ($p = 0.044$). These varying results also tell us that the performance of our KNN model depends heavily on the dataset.

The reduction is nearly free. Edited and Condensed KNN seemingly never differ too much from full KNN. There is only one exception: Condensed KNN on Car Evaluation, which is dramatically worse (0.070 vs 0.051, $p = 0.044$) despite keeping 42.5 percent of its points. The Breast Cancer data is the best case, with Condensed KNN keeping 10.5 percent of the data with no true accuracy loss ($p = 0.329$).

Edited KNN seems to be the weakest method. It is time after time worse than full KNN on Abalone ($p = 0.045$) and Forest Fires ($p = 0.012$); both of these times it discards most

of the set (14.6 percent and 28.5 percent). Condensed KNN never seems to lose to full KNN to much on any of the datasets and seems to beat Edited KNN dramatically on the same two datasets ($p = 0.042$, $p = 0.030$).

Condensed KNN is the most reliable reducer overall. Edited KNN’s failures cluster a lot on the regression task where it seems to edit too aggressively for continuous targets.

6 Conclusion

In this project, we evaluated several K-Nearest Neighbor (KNN) algorithms across three classification and three regression datasets. Our goal was to compare standard KNN, Edited KNN, and Condensed KNN, along with testing the effects hyperparameter tuning has on their performance. Using our 5x2 cross-validation harness, we found the effectiveness of each KNN variant was extremely dependent on the dataset as well as the task.

We found that overall, the KNN variants generally out performed the Null Models, especially on the classification datasets. However, in the Forest Fire dataset, we found that KNN does not always provide an advantage when predicting, as the Null Model gave better results than all of the KNN variants on this dataset. This provides us the support we need for our first hypothesis, while also showing us the effectiveness of KNN depends on the structure of a datasets features.

Our test with hyperparameter tuning returned mixed results. Tuning greatly improved the results of some datasets, while producing worse for others. This means our second hypothesis received partial support. We also observed that across each dataset, Edited KNN and Condensed KNN had observable differences. This gives support to our third hypothesis, that the two reduction methods would result in a noticeable difference in performance.

The results we found, demonstrate that there is no single KNN approach that performs the best across all datasets. The effectiveness of KNN, hyperparameter tuning, and instance reduction all depend on the characteristics of the dataset features and the relationship between those features and the target variable.

7 Appendix

7.1 Main Lessons Learned

The reduction methods tend to behave very differently depending on the task and dataset. Condensed KNN performed very well on most of the datasets except for the Abalone and Forest Fires data sets. Edited KNN did very poorly on the two regression datasets (Abalone, Forest Fires). Edited KNN’s leave-one-out removal rule, which is tuned for classification correctness, seems to be too aggressive when correctness is a constant error threshold. It can strip out points that were actually informative, not just noise. This taught us that the training set doesn’t necessarily preserve the information needed for predictions.

Breast Cancer’s Condensed KNN kept about 10 percent of the training data with no statistically significant difference in performance. That is a very useful result if you were trying to justify the value of instance reduction. However, Car Evaluation shows that this is not guaranteed because the reduction cost real accuracy, and it took significance testing to confirm it was not noise. This shows us that, overall, the effectiveness of instance reduction depends on how the datasets are structured.

The statistical testing changes conclusions that you would draw from raw numbers. Computer Hardware's Edited KNN MSE looked almost double the KNN's data. The 5x2cv showed that gap was not statistically significant, and the dataset was just too small for the comparison to be reliable. This showed us that differences in average performance should be considered using statistical significance and not from looking at raw values alone.

Forest Fires is a dataset that does not have a learnable distance structure, and KNN never seemed to beat the null model. That is a very real conclusion because of how extremely skewed and mostly zero-targeted the data is, but it is not an error of implementation. The usefulness of the distance relationship depends on whether or not similar observations provide useful information for target prediction.

7.2 Delineation of who did what on the project

Zach worked on: Data Acquisition and the preprocessing, Core building blocks, Null Models, the 5x2 cross-validation harness and the built in hyperparameter tuner.

Gaige worked on: Hyperparameter tuning, distances metrics, Edited KNN, Condensed KNN, KNN, KNN Full, Results Analysis, and implementation of all datasets.

Both worked on: Testing and debugging, experimental evaluation, paper writing, and interpretation of results.

References

Peter Hart. The condensed nearest neighbor rule. *IEEE Transactions on Information Theory*, 14(3):515–516, 1968.

Dennis L Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, (3):408–421, 1972.